

**LAPORAN PRAKTIKUM**  
**ALGORITMA DAN PEMROGRAMAN 2**  
**MODUL 17**  
**“SKEMA PEMROSESAN SEKUENSIAL”**



**DISUSUN OLEH:**  
**SALSADILLA HANNY AZIZAH**  
**108092500014**  
**S1IF-13-02**  
**DOSEN:**  
**Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**  
**2024/2025**

## DASAR TEORI

Deret merupakan penjumlahan dari suku-suku suatu barisan bilangan. Dalam matematika, deret dapat terdiri atas sejumlah suku tertentu maupun tak hingga. Konsep deret banyak digunakan dalam berbagai bidang, salah satunya untuk melakukan pendekatan terhadap suatu nilai tertentu melalui proses penjumlahan yang dilakukan secara berulang. Semakin banyak suku yang dijumlahkan, maka hasil yang diperoleh umumnya akan semakin mendekati nilai sebenarnya.

Salah satu deret yang digunakan untuk menentukan pendekatan nilai konstanta  $\pi$  adalah deret Leibniz. Deret ini dinyatakan sebagai  $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$  dan seterusnya. Setiap suku pada deret memiliki tanda positif dan negatif secara bergantian. Dengan menjumlahkan suku-suku tersebut dan mengalikan hasilnya dengan empat, maka akan diperoleh nilai pendekatan  $\pi$ . Semakin banyak jumlah suku yang digunakan, maka hasil yang diperoleh akan semakin mendekati nilai  $\pi$  yang sebenarnya.

Dalam implementasinya, perhitungan nilai  $\pi$  juga dapat dilakukan dengan memanfaatkan bilangan acak. Bilangan acak merupakan bilangan yang dibangkitkan secara acak pada rentang tertentu. Pada bahasa pemrograman Go, bilangan acak dapat dihasilkan menggunakan package `math/rand`, salah satunya melalui fungsi `rand.Float64()`, yang menghasilkan bilangan riil acak pada interval 0 sampai 1. Bilangan acak tersebut banyak dimanfaatkan dalam simulasi dan pemodelan matematika.

Salah satu metode yang memanfaatkan bilangan acak adalah metode Monte Carlo. Metode Monte Carlo merupakan metode numerik yang menggunakan proses simulasi secara berulang untuk memperoleh pendekatan terhadap suatu nilai tertentu. Semakin banyak data yang digunakan dalam simulasi, maka tingkat ketelitian hasil yang diperoleh akan semakin baik. Metode ini banyak digunakan untuk menyelesaikan berbagai permasalahan yang berkaitan dengan probabilitas, statistika, maupun pendekatan numerik.

Pada simulasi penaburan topping pizza, sebuah lingkaran dengan jari-jari  $r$  ditempatkan di dalam sebuah wadah berbentuk persegi dengan panjang sisi  $2r$ . Luas lingkaran dinyatakan dengan persamaan  $\text{Luas Pizza} = \pi r^2$ , sedangkan luas persegi dinyatakan dengan  $\text{Luas Wadah} = (2r)^2 = 4r^2$ . Perbandingan antara luas lingkaran dan luas persegi menghasilkan nilai  $\pi/4$ . Dengan membangkitkan titik-titik secara acak pada bidang persegi dan menghitung banyaknya titik yang berada di dalam lingkaran, maka nilai  $\pi$  dapat diperkirakan menggunakan metode Monte

Carlo. Semakin banyak titik yang digunakan dalam simulasi, maka nilai  $\pi$  yang diperoleh akan semakin mendekati nilai sebenarnya.

## SOAL LATIHAN

### Statement

#### 1. Rerata Marker

### Source Code:

```
package main

import "fmt"

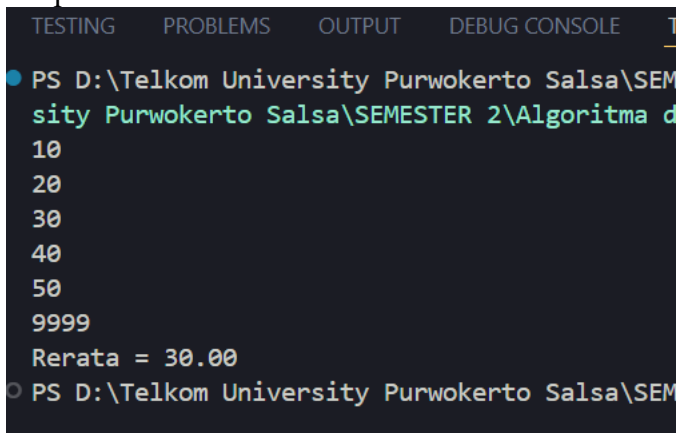
func main() {
    var bil, jumlah, rata float64
    var banyak int

    fmt.Scan(&bil)

    for bil != 9999 {
        jumlah += bil
        banyak++
        fmt.Scan(&bil)
    }

    if banyak > 0 {
        rata = jumlah / float64(banyak)
        fmt.Printf("Rerata = %.2f\n", rata)
    } else {
        fmt.Println("Tidak ada data")
    }
}
```

### Output:



```
TESTING  PROBLEMS  OUTPUT  DEBUG CONSOLE  T
● PS D:\Telkom University Purwokerto Salsa\SEM
sity Purwokerto Salsa\SEMESTER 2\Algoritma d
10
20
30
40
50
9999
Rerata = 30.00
○ PS D:\Telkom University Purwokerto Salsa\SEM
```

### Deskripsi Program:

Program ini dibuat untuk menghitung rata-rata dari beberapa bilangan real yang dimasukkan oleh pengguna. Input akan terus dibaca sampai pengguna memasukkan

angka 9999 sebagai penanda akhir. Setelah semua data terbaca, program akan menghitung jumlah seluruh bilangan kemudian membaginya dengan banyaknya data yang dimasukkan sehingga diperoleh nilai rata-ratanya.

## 2. Pencarian String dalam Sekumpulan String

### Source Code:

```
package main

import "fmt"

func main() {
    var x, data string
    var n, i int
    var jumlah int
    var posisi int
    var ditemukan bool

    fmt.Scan(&x)
    fmt.Scan(&n)

    posisi = -1

    for i = 1; i <= n; i++ {
        fmt.Scan(&data)

        if data == x {
            jumlah++
            ditemukan = true

            if posisi == -1 {
                posisi = i
            }
        }
    }

    if ditemukan {
        fmt.Println("String ditemukan")
        fmt.Println("Posisi pertama:", posisi)
    } else {
        fmt.Println("String tidak ditemukan")
    }

    fmt.Println("Jumlah kemunculan:", jumlah)

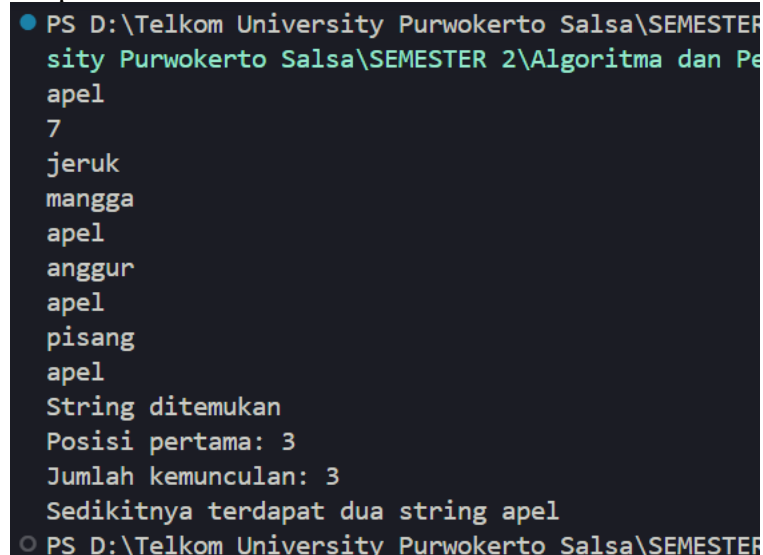
    if jumlah >= 2 {
        fmt.Println("Sedikitnya terdapat dua string", x)
    } else {
```

```

        fmt.Println("Kurang dari dua string", x)
    }
}

```

Output:



```

PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan Pe
ity Purwokerto Salsa\SEMESTER 2\Algoritma dan Pe
apel
7
jeruk
mangga
apel
anggur
apel
pisang
apel
String ditemukan
Posisi pertama: 3
Jumlah kemunculan: 3
Sedikitnya terdapat dua string apel
PS D:\Telkom University Purwokerto Salsa\SEMESTER

```

Deskripsi Program:

Program ini digunakan untuk mencari apakah suatu string *x* terdapat pada kumpulan string yang diberikan. Selain mengecek keberadaan string tersebut, program juga menentukan posisi pertama ditemukannya string *x*, menghitung jumlah kemunculannya, serta memeriksa apakah string *x* muncul minimal dua kali dalam kumpulan data yang dimasukkan.

### 3. Curah Hujan pada Daerah A, B, C, dan D

Source Code:

```

package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    var x, y float64
    var A, B, C, D int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        x = rand.Float64()

```

```

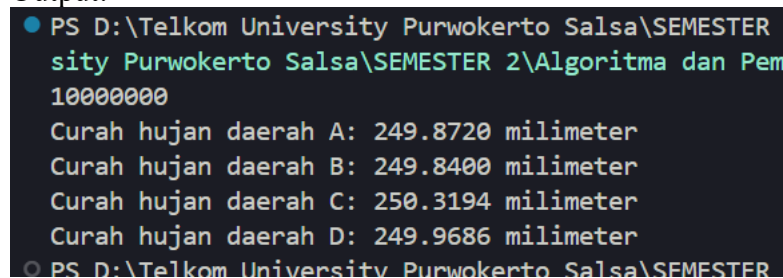
    y = rand.Float64()

    if x < 0.5 && y < 0.5 {
        A++
    } else if x >= 0.5 && y < 0.5 {
        B++
    } else if x >= 0.5 && y >= 0.5 {
        C++
    } else {
        D++
    }
}

fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(A)*0.0001)
fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(B)*0.0001)
fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(C)*0.0001)
fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", float64(D)*0.0001)
}

```

Output:



```

PS D:\Telkom University Purwokerto Salsa\SEMESTER
ity Purwokerto Salsa\SEMESTER 2\Algoritma dan Pem
10000000
Curah hujan daerah A: 249.8720 milimeter
Curah hujan daerah B: 249.8400 milimeter
Curah hujan daerah C: 250.3194 milimeter
Curah hujan daerah D: 249.9686 milimeter
PS D:\Telkom University Purwokerto Salsa\SEMESTER

```

Deskripsi Program:

Program ini digunakan untuk mensimulasikan tetesan hujan yang jatuh pada empat daerah, yaitu A, B, C, dan D. Titik jatuh setiap tetesan dibuat secara acak menggunakan fungsi `rand.Float64()`. Setelah semua tetesan diproses, program akan menghitung jumlah tetesan pada masing-masing daerah dan mengubahnya menjadi nilai curah hujan dalam satuan milimeter.

#### 4. Formula Leibniz

Source Code:

```

package main

import "fmt"

func hitungSuku(i int) float64 {
    var pembilang float64
    var penyebut float64

```

```

        if i%2 != 0 {
            pembilang = 1.0
        } else {
            pembilang = -1.0
        }

        penyebut = float64(2*i - 1)

        return pembilang / penyebut
    }

func hitungPiDeret(n int) {
    var totalDeret float64 = 0.0
    var hasilPi float64
    var i int

    for i = 1; i <= n; i++ {
        totalDeret = totalDeret + hitungSuku(i)
    }

    hasilPi = 4.0 * totalDeret

    fmt.Printf("Hasil Pi: %.7f\n", hasilPi)
}

func main() {
    var n int

    fmt.Print("N suku pertama: ")
    fmt.Scan(&n)

    hitungPiDeret(n)
}

```

Output:

```

TESTING  PROBLEMS  OUTPUT  DEBUG CONSOLE

● PS D:\Telkom University Purwokerto Sals...
  sity Purwokerto Salsa\SEMESTER 2\Algor...
  N suku pertama: 1000000
  Hasil PI: 3.1415917
  Hasil PI: 3.1415726536
  Pada i ke: 50000
○ PS D:\Telkom University Purwokerto Sal...

```



#### Deskripsi Program:

Program ini dibuat untuk menghitung nilai pendekatan  $\pi$  menggunakan deret Leibniz. Pada tahap pertama, program menerima input berupa banyaknya suku pertama yang akan dihitung, kemudian menjumlahkan suku-suku deret secara bergantian antara positif dan negatif hingga mencapai jumlah suku yang ditentukan. Hasil penjumlahan tersebut kemudian dikalikan dengan empat sehingga diperoleh nilai pendekatan  $\pi$ . Selanjutnya, program dimodifikasi untuk melakukan perhitungan secara berulang mulai dari suku pertama dan akan berhenti apabila nilai suku berikutnya sudah lebih kecil atau sama dengan 0,00001. Setelah kondisi tersebut terpenuhi, program menampilkan nilai pendekatan  $\pi$  yang diperoleh beserta nilai  $i$  saat proses perhitungan dihentikan.

## 5. Kedai Pizza

#### Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n, toppingPizza int
    var x, y float64

    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

    for i := 1; i <= n; i++ {
        x = rand.Float64()
        y = rand.Float64()

        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            toppingPizza++
        }
    }

    fmt.Println("Topping pada Pizza:", toppingPizza)
}
```

Output:

```
TESTING  PROBLEMS  OUTPUT  DEBUG CONSOLE

● PS D:\Telkom University Purwokerto Salsa\S
  sity Purwokerto Salsa\SEMESTER 2\Algoritma
  Banyak Topping: 1234567
  Topping pada Pizza: 969175
○ PS D:\Telkom University Purwokerto Salsa\S
```

### Deskripsi Program:

Program ini digunakan untuk menghitung banyaknya topping yang jatuh tepat pada pizza. Program menerima input berupa banyak topping yang akan ditaburkan, kemudian membangkitkan titik acak (x,y) pada interval 0 sampai 1. Jika titik tersebut berada di dalam lingkaran dengan pusat (0,5;0,5) dan jari-jari 0,5, maka topping dianggap berada pada pizza.

### Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n, toppingPizza int
    var x, y, pi float64

    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

    for i := 1; i <= n; i++ {
        x = rand.Float64()
        y = rand.Float64()

        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            toppingPizza++
        }
    }

    pi = 4 * float64(toppingPizza) / float64(n)

    fmt.Println("Topping pada Pizza:", toppingPizza)
    fmt.Printf("PI : %.10f\n", pi)
}
```

Output:

```
TESTING PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan F
sity Purwokerto Salsa\SEMESTER 2\Algoritma dan F
Banyak Topping: 1234567
Topping pada Pizza: 97000
PI : 3.1428022942
PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan F
sity Purwokerto Salsa\SEMESTER 2\Algoritma dan F
Banyak Topping: 10
Topping pada Pizza: 8
PI : 3.2000000000
PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan F
sity Purwokerto Salsa\SEMESTER 2\Algoritma dan F
Banyak Topping: 256
Topping pada Pizza: 199
PI : 3.1093750000
PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan F
sity Purwokerto Salsa\SEMESTER 2\Algoritma dan F
Banyak Topping: 5000
Topping pada Pizza: 3903
PI : 3.1224000000
PS D:\Telkom University Purwokerto Salsa\SEMESTER 2\Algoritma dan F
```

Deskripsi Program:

Program ini merupakan pengembangan dari program sebelumnya. Selain menghitung banyak topping yang berada pada pizza, program juga menghitung nilai pendekatan konstanta  $\pi$  berdasarkan perbandingan antara jumlah topping yang berada pada pizza dengan jumlah seluruh topping yang ditaburkan. Titik-titik topping dibangkitkan secara acak menggunakan fungsi `rand.Float64()`, kemudian program menentukan apakah titik tersebut berada di dalam lingkaran atau tidak. Setelah semua titik diproses, program menampilkan jumlah topping yang berada pada pizza beserta nilai pendekatan  $\pi$  yang diperoleh.



## DAFTAR PUSTAKA

Modul Praktikum Algoritma dan Pemrograman 2, Modul 17: *Skema Pemrosesan Sekuensial*.  
Fakultas Informatika, Telkom University.

Weisstein, Eric W. *Leibniz Formula*. Wolfram MathWorld.  
<https://mathworld.wolfram.com/LeibnizFormula.html>

Wikipedia. *Deret (Matematika)*. [https://id.wikipedia.org/wiki/Deret\\_\(matematika\)](https://id.wikipedia.org/wiki/Deret_(matematika))

Wikipedia. *Monte Carlo Method*. [https://en.wikipedia.org/wiki/Monte\\_Carlo\\_method](https://en.wikipedia.org/wiki/Monte_Carlo_method)

Dokumentasi Go. *Package math/rand*. <https://pkg.go.dev/math/rand>